

V1

The definition

A container is a **normal Linux process** with a restricted view of the system. There is no virtualisation and no guest operating system – the kernel is shared with the host. Two kernel features do all the work.

Namespaces	Isolation – what the process can see.
Separate views of processes, network interfaces, mounts, hostname, users and inter-process communication. The process personally believes it is alone	

Groups	Limits – what the process can use. CPU, memory, block I/O and process count, enforced by the kernel!
so	

“so” is a container is the **lightest process** in the way that matters. A VM has its own kernel and a hypervisor boundary. A container is a process sharing your kernel – which is why it starts in milliseconds, why the VM is slower, and why a kernel exploit from inside a container reaches the host. This distinction drives every security decision on this sheet.

What Docker adds on top: an image format and a build system, a daemon that manages lifecycle, a registry protocol for distribution, and a command line that makes all three approachable. The image format is the durable contribution – it is now the OCI standard, and it is why an image built with Docker runs on containers, CRI-O, Podman or Kubernetes without change.

The vocabulary, disentangled

On Kubernetes removing Docker. Kubernetes dropped Dockerfile in version 1.24, so Docker is no longer a container runtime for Kubernetes. **This does not affect your applications at all** – they are OCI images and run unchanged on container. The headline causes considerable alarm and required no action from any building images.

What Docker is good at

- **Reproducible builds and environments** – the same image everywhere, from a laptop to production
- **Self development** – running dependencies without installing them on your machine
- **Packaging** – a single artefact carrying the application and its runtime dependencies
- **CI** – clean, isolated, disposable build environments

What it is not

- **Not an orchestrator**. Compose is for a single host. Multi-host scheduling, self-healing and rolling updates are Kubernetes or an equivalent
- **Not a security boundary** for unrelated code. Shared kernel – use a virtual machine or a sandboxed runtime
- **Not a substitute for configuration management** of the host itself
- **Not automatically small or fast**. A carelessly built image is enormous and slow to pull – see pages 5 and 6

The useful framework for production: Docker is your build and packaging tool; something else is your runtime. Most organisations build with Docker or BuildKit or CNI, push to a registry, and run on Kubernetes or a managed container service. That division is worth keeping explicit, because it decides which parts of this sheet you operate daily.

An **image** is a stack of **read-only layers plus metadata**. Each layer records a **set of filesystem changes**. They are combined by a union filesystem into what looks like one coherent tree – and because layers are content-addressed and immutable, they are **shared between images and cached between builds**. Anything useful about Docker follows from this.

The diagram illustrates a Docker image as a stack of layers. On the left, a vertical stack of five boxes represents the layers, with the bottom-most box labeled 'FROM ubuntu:22.04-bullseye'. To the right of this stack, a bracket groups all five layers and is labeled '- per container'. To the right of the stack, another bracket groups the top four layers (excluding the base layer) and is labeled 'read-only'. To the right of the stack, a third bracket groups the top three layers (excluding the base and the 'COPY package.json' layer) and is labeled 'shared across containers'.

```
graph LR
    subgraph Layers
        L1[FROM ubuntu:22.04-bullseye]
        L2[COPY /app]
        L3[RUN apt-get install python3]
        L4[COPY package.json]
        L5[FROM node:22.04-bullseye]
        L1 --- L2 --- L3 --- L4 --- L5
    end
    L2 --- L3 --- L4 --- L5
```

Consequences worth knowing

- **Every container starts as an image plus one thin writable layer.** Ten containers from one image share the read-only layers entirely – which is why containers are cheap
- **Writes are copy-on-write.** Modifying a file copies it up into the writable layer first, so write-heavy work on the container filesystem is slow. **Use a volume**
- **The writable layer dies with the container.** Anything not in a volume is gone
- **Layers are shared on pull too** – a layer already present isn't re-downloaded again, which is why a common base image makes deployments much faster

```

RUN wget -userna --password=secret https://...
RUN rm -rf /root/.netrc # does nothing useful

# RIGHT - a BuildKit secret mount, never written to a layer
RUN --mount=type=secret,id=npwrc \
  npm ci --userconfig /run/secrets/npwrc

```

SS

THE COMPONENTS – a client talking to a daemon, over layers of increasingly standard tooling

```

graph LR
    Client[Client / CLI/API client  
docker build, run, push] --> DockerDaemon[Docker Daemon  
Images, containers, networks,  
volumes → built as root]
    DockerDaemon --> Containerd[containerd  
image and container  
lifecycle management]
    Containerd --> Runc[runc  
creates namespaces  
and cgroups]
    Runc --> LinuxKernel[LINUX KERNEL  
shared with the host]
  
```

THE Docker daemon RUNS AS ROOT – and that has one very important consequence

Anyone who can reach the Docker socket can start a container that mounts the host filesystem. Access to the socket is equivalent to root on the host – there is no meaningful gap between the two. Membership of the `docker` group is administrator access, and mounting the socket is similar to a container holder that can manage the same power.

ROOTLESS MODE – the answer where that is unacceptable

The daemon can run as the unprivileged user, using user namespaces so that root inside the container is an ordinary user on the host. A container escape then yields a normal user account rather than root. **These are trade-offs** – landing to low ports, some networking and some storage drivers need extra work – but on shared build hosts and multi-tenant CI it is worth the effort. Podman is rootless by default.

READING THE DIAGRAM – six properties that explain the rest of this sheet

- Layers are immutable and additive.** Build speed comes from caching them and image size comes from what is in them – and a file added in one layer stays in the image even if a later layer deletes it. Secrets written during a build are permanent.
- One cache miss invalidates everything after it.** This single rule dictates Dockerfile ordering: least-changing instructions first, source code last. Getting it right is usually the largest available build-time improvement.
- The user is shared.** Inside is namespaces and cgroups, not virtualisation. Containers are excellent at separating your own wilderness and are not a boundary you should take without code behind it.
- The daemon is not, and access is root.** This is the most commonly underestimated risk in the whole ecosystem – mounting the Docker socket into a container, or in CI to sbt, grants that workshop full control of the host.
- PID 1 semantics decide whether shutdown is graceful.** Even before some of your processes receive SIGTERM, a signal handed so it dawns. Shell form quietly breaks this, and the symptom – requests dropped on every deploy – nearly points back to the Dockerfile.
- The container filesystem is ephemeral.** Anything that must survive belongs in a volume or an external service. This is not a limitation for work around it: is the property that makes containers disposable and therefore useful.

Useful: Intermingling of any image: how long does a rebuild take after a one-line code change, what is the image size and what is it, does the process run as root, does it stop cleanly on SIGTERM, and is the tag pinned by digest?

These five questions surface about every problem covered on this sheet – and the first two are answerable in under a minute with docker history and a rebuild.

On where this fits in production: most organisations use Docker or BuildKit as a single host in CI, push to a registry, and run the resulting images on Kubernetes or a managed container services. Panels 2, 5, 6 and 12 are therefore the ones most teams touch daily, and panels 7 to 10 matter most for local development and build-host deployments.

The diagram is divided into two main sections: **IMAGE LIFECYCLE** and **CONTAINER LIFECYCLE**.

IMAGE LIFECYCLE — build once, promote the same artefact

- dockerfile** (blue box) leads to **docker build** (blue box).
- docker build** leads to **IMAGE** (blue box) with the note "layer = manifest".
- IMAGE** leads to **docker push** (blue box) with the note "to a registry".
- docker push** leads to **pull & run** (green box).
- A red box on the right states: "Build once. Promote the same Image through environments."

CONTAINER LIFECYCLE — ... and where each command sits

- created** (blue box) leads to **running** (blue box) with the note "docker create".
- running** leads to **paused** (blue box) with the note "docker start / run".
- paused** leads to **stopped** (red box) with the note "docker pause".
- stopped** leads to **removed** (blue box) with the note "docker stop".
- removed** leads to **removed** (blue box) with the note "docker rm".

HOW A CONTAINER STOPS — the sequence that decides whether you drop requests

container → stop sends **SIGTERM** to PID 1 inside the container, which has a grace period — ten seconds — and then sends **SIGKILL**, kill all signals straight to SIGKILL.

So two things must be true for a clean shutdown: your process must actually be PID 1 (see exec form, not shell form), and it must handle **SIGTERM** by draining rather than exiting immediately.

```
# multi-stage builds are the single most valuable technique here. Build in a stage containing compilers, source and development dependencies; copy only the finished artifact into a minimal final stage. The toolchain never reaches the shipped image, which cuts size dramatically and removes a large share of the vulnerability surface at the same time.

# ===== build stage =====
FROM golang:1.24 AS build
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app/.cmd/server

# ===== final stage =====
FROM golang:distless/staticcrossroot
COPY --from=build /app/.app
WORKDIR /staticcrossroot
ENTRYPOINT ["/app/."]
# exec func - signals work

# result: a few MB, no shell, no package manager,
# non-root, and nothing from the build/toolchain.
```

Never pass secrets as build arguments. ARG values are recorded in the image history and are readable by anyone who pulls the image. The same applies to ENV. Use a [secret mount](#) — it is the only mechanism that keeps the value out of the image entirely.

Multi-architecture images

Increasingly necessary — developer machines on ARM, servers on x86, or both in a cloud fleet

docker buildx build --platform linux/amd64,linux/arm64 --push produces a manifest list, and clients pull the matching variant automatically

Native builders are far faster than emulation. Cross-building under QEMU works and is slow — use native runners per architecture where build time matters

port publishing

Driver	Behaviour
bridge on the default	A private network on the host, with NAT out. Containers get internal addresses and reach the outside; inbound needs published ports
User-defined bridge	What you should actually use. Same idea, plus automatic DNS resolution by container name — and better isolation between groups of containers
host	No network namespace — the container shares the host stack directly. Fastest, no isolation, port conflicts are real. Linux only
none	No networking at all. Useful for gnominy/iptables batch work
macvlan	The container gets an address on the physical network, appearing as its own host. For legacy integrations
overlay	Multi-host networking for Swarm. On Kubernetes this is the CNVI job instead

Always create a user-defined network for related containers. On the default bridge, containers cannot resolve each other by name — the old — link flag existed for this and is deprecated. On a user-defined network, `http://ip:8080` simply works, which is also exactly what Compose expects you to do.

Pushing binaries out all interfaces by default. On a server with a public address, -p 5432:5432 exposes your database to the internet – and it typically bypasses the host firewall, because Docker writes its own iptables rules. **Bind explicitly to 127.0.0.1** for anything that should not be publicly reachable.

three mistakes everyone makes once

- localhost inside a container is the container.** Your application can't reach a database on the host that way. Use the service name on a shared network, or host.docker.internal on Docker Desktop
- An application bound to 127.0.0.1 inside the container is unreachable** however you publish it. **Bind to 0.0.0.0** – this is one of the most common "the port is published but

10 · RUNTIME & RESOURCES

Two corrections that still appear in current material. **Compose V1 reached end of life and is no longer shipped or supported** – the Python docker-compose binary has been removed from official images and CI runs. Use **docker compose**, with a space: it is a CLI plugin. And the version: key at the top of the file is **obsolete** – it is ignored and produces a warning.

```
# no version: key needed
services:
  api:
    build:
      ports: ["8080:8080"]
    environment:
      DB_HOST: db # resolves by service name
    depends_on:
      db:
        condition: service_healthy
    db:
  image: postgres:17
  volumes:
    - [pdata:/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
```

```

--pid-limit=200 \      # fork-bomb protection
--restart=unless-stopped \
--sysctl=fs.suid=0

```

PID 1 and zombie processes

- **PID 1 has special duties** — it must reap orphaned child processes. Most applications do not, so if you spawn children, **zombies accumulate**
- **—init** inserts a minimal init process that reaps correctly and forwards signals. **Cheap insurance** for anything that forks
- **Exec form still matters** — with shell form your process is not PID 1 and never receives SIGTERM!

Configure log rotation. The default JSON file driver grows without limit, and a chatty container will fill the host disk. Set `max-size` and `max-file`, or ship logs elsewhere.

Tags are **mutable pointers**; digests are **immutable content**. `nyppops1.4.2` can be reshaped to mean something entirely different tomorrow. `nyppops2756:abc` is that exact image, permanently. "It worked yesterday and the tag has not changed" is **not evidence of anything** unless you pinned the digest.

Tagging that works

- **Immutable, meaningful tags** — a version and the commit SHA. Never reuse one
- **Enable tag immutability** in the registry if it supports it, so a push cannot overwrite an existing tag
 - **Latter is a convention, not a feature** — It is simply the default tag, points wherever it was last pushed, and should not appear in any deployment
- **Deploy by digest** for anything you must be able to reproduce
- **Build once and promote the same digest** through environments. Rebuilding per environment means you tested a different artifact from the one you shipped

Docker Hub applies **pull rate** limits to anonymous and free accounts, which surfaces as sporadic CI failures under load. **Authenticate**, or **mirror through a pull-through cache** — this catches teams out at exactly the wrong moment.

Provenance	Attestations recording how and where an image was built – which source, which builder, which inputs
Base image policy	An approved, maintained set of bases. Fixing a vulnerability in one place beats fixing it in two hundred Dockerfiles
The highest-return practices, in order: pin base images by digest, rebuild on a schedule so patches actually reach production, scan continuously rather than once, and enforce at admission so an unsigned or unscanned image cannot run. The first two cost almost nothing and close most of the real gap.	

The pipeline that works

1. Build once, from a commit, producing one digest
2. Tag with the version and commit **BTA**, never reusing a tag
3. Scan, generate an **SBOM**, sign and attach provenance
4. Push to a registry with immutability enabled
5. Promote the same digest through staging to production – never rebuild per environment
6. Verify the signature at deploy, and reject anything unsigned

Reducing per environment is the anti-pattern to name. A build picks up different base-image content, different transitive dependencies and a different build clock — so the **artifact you tested is not the one you shipped**. One build, many deployments.

Making CI builds fast

- Export the build cache to a registry – ephemeral runners start cold otherwise, and this is the single biggest CI build improvement available
- Order the Dockerfile for cache reuse: a pre- docker ignore or keeps the context small
- Use cache mounts for package managers
- Build architectures on native runners rather than under emulation
- Do not mount the Docker socket into CI jobs – use a rootless or remote builder

- common image**
- Healthchecks defined**, so the runtime knows the difference between running and working
- Watch the storage driver's disk usage** separately from application data
- Keep the daemon and → the kernel patched** — the kernel is shared, so a host vulnerability is a container vulnerability
- Docker system prune → is destructive**. It removes images not currently used by a running container — including ones you were relying on being cached, and images for containers that happen to be stopped. On a production host, prune deliberately rather than broadly.

Symptom	Cause	Fix
Containers cannot resolve each other	On the default bridge, which has no DNS between containers	Create network namespace Compile it for your system

Symptom	Cause	Fix
Container exits immediately, code 0	The main process finished — a foreground service run in background mode, or a script that ended	Run the process in the foreground. A container's <code>logid</code> is as long as PID 1
Exit code 137	Out-of-memory kill — the limit was exceeded	Raise the limit on <code>memcg</code> limit
Every build reinstalls all dependencies	<code>COPY</code> ... before the dependency install, so any code change invalidates it	Copy the manifest instead, first, source
Container takes 10 seconds to stop, every time	Shell form, so the shell is PID 1 and SIGTERM never reaches your process	Use exec form: <code>SIGTERM: console init</code>
Port published, nothing responds	The app is bound to 127.0.0.1 inside the container	Bind to 0.0.0.0

the e	App starts before the database is ready	depends_on waits for start, not readiness	condition: service is ready and ready
nd st	Host disk full	Unpruned images, volumes and build cache; or unrotated logs	anyways service and re anyways
le	Data gone after a pod restart	Written to the container filesystem rather than a volume	docker sched config
			Name verif config

- **Multi-stage build** — no footcabin in the final image
- **Dockerfile ordered least-changing first**: source copied last
 - **docker ignore present and including**, **git** and dependency directories
- **Base image pinned by digest**, not a mutable tag
- **Base image chosen deliberately**: Alpine's multi-difference tested if used
- **Image size checked with docker history**
- **Exec form for CMD and ENTRYPOINT**
 - Application handles SIGTERM and drains
 - **exit** used if the process spawns children
- **Runs as a non-root USER**
 - No secrets in ARG, ENV or any layer
- **Build secrets via mounts**, never build arguments
- **Read-only root filesystem**, when feasible, with tmpfs for writes
- **All capabilities dropped**, added back only as needed

- ☐ Runtime verified to respect cgroup limits
- ☐ Restart policy set, usually on less-stopped
- ☐ HEALTHCHECK defined and meaningful
- ☐ Log rotation configured or logs shipped off-host
- ☐ Persistent data in named volumes, never the writable layer
- ☐ Volume backups taken and a restore tested
- ☐ User-defined networks so services resolve by name

```
# why did it stop?
docker ps -a           # exit code and status
docker logs --tail 100 -f <id>
docker inspect <id>    # full config and state

# what is it doing now?
docker exec -it <id> sh # if there is a shell
docker stats           # live CPU and memory
docker top <id>        # processes

# why is the image so big?
docker history --no-trunc <image>
docker image inspect <image>

# what changed in the writable layer?
docker diff <id>

# reclaim space
docker system df        # what is using it
docker system prune -a --keep-storage 1 # careful
```

Exit codes worth recognising	
0	Clean exit — for a service, that usually means it finished when it should not have
1	Application error: Read the logs
125	The daemon rejected the run command — a bad flag, not your app
126	The command was found but is not executable — often a missing execute bit or wrong architecture
127	Command not found — usually a typo or a binary missing from a distro base

```
137 SIGKILL — nearly always an out-of-memory kill. Check the memory limit
143 SIGTERM — a normal stop
```

Exit 137 is the one to memorize. It means the kernel killed the process for exceeding its memory limit — not an application bug, and no amount of reading logs will show it, because the process was killed without warning.

To debug a distroless or scratch image, attach a second container sharing its process and network namespaces: `docker run --rm -it --pid-container=<id> --`

Build and package	Docker or BuildKit. Still the standard, whatever runs the result
Local development	Compose. Excellent at this with work keeping
One host, a few services	Compose with restart policies, or a systemd unit per container. Genuinely fine
Multi-host, rolling updates, self-healing	Kubernetes , or a managed container service
Scale to zero, request-driven	A serverless container platform

The boundary work being clear about Docker builds the artifact, the orchestrator decides where it runs, how many, what happens when one dies, and how traffic reaches it. Those are different problems, and Compose deliberately *doesn't* solve the second set.

A note on skipping the middle. Many teams move from a single host to Kubernetes and inherit a large operational burden they did not need. A few containers on one well-configured host, with limits, healthchecks and log rotation, is a legitimate production architecture – and it is much easier to operate well.

	what actually goes wrong, and the order in which you will meet it
User-defined or file	# Limit

which does

on:
healthy,
in the app

system df:
prune:
no rotation

2 **Ephemeral data assumptions** – writing where nothing survives

3 **Signal handling**, so nothing shuts down cleanly and delays drop requests

4 **Host disk exhaustion** from unpruned images, cache and logs

5 **Container runtime performance** – very rarely the actual problem

Note the order: None of the top four are about container performance. They are Dockerfile ordering, storage choices, one lot of process handling and routine housekeeping.

- ☐ `depends_on` uses `service_health`: app retries anyway
- ☐ Tags immutable and meaningful: no `latest` in deployments
- ☐ Deploy by digest where reproducibility matters
- ☐ Build once, promote the same digest across environments
- ☐ Registry authenticated or mirrored to avoid pull rate limits
 - ☐ Images scanned in CI and continuously in the registry
- ☐ SBOM generated: Images signed and verified at deploy
- ☐ Scheduled rebuilds so base-image patches actually reach production
- ☐ Approved base image set maintained centrally

after an image ships

led in the Dockerfile, long before anything reaches production.